

Lab Week 3 Manual

Race Conditions, Monitors, Locks & Reentrancy

Dr. Mohamad AOUDÉ

May 11, 2026

Course: Concurrency & Distributed Systems (ULFG SEM VIII)

Week: 3 – Locks, Monitors & Reentrancy

Lab: week3-lab / week3-day1-race-monitors-lock-identity

Canonical Location: <https://github.com/maoude/concurrency-course-complete>

Learning Objectives

Upon completing this lab, you will:

- Understand *race conditions* — how lost updates occur under concurrent access to shared mutable state
- Master *mutual exclusion* using Java’s intrinsic monitors (`synchronized`)
- Distinguish between *lock identity* and *lock scope* — instance locks vs static locks
- Recognize thread *states* (RUNNABLE, BLOCKED, WAITING) and how coordination primitives affect them
- Apply *explicit locks* (`ReentrantLock`) and `try-finally` patterns
- Design thread-safe invariants using lock-ordering discipline (transfer protocol)
- Implement *producer-consumer* patterns with `wait()` / `notifyAll()`

Lab Structure Overview

Part	Topic	Demos	Exercises
1	Race Conditions	Demo01, Demo02	Ex1, Ex2
2	Monitors	Demo03-Demo07	Ex1, Ex2
3	Lock Identity	(in Part 1-2 context)	Ex1, Ex2
4	Thread States	(BLOCKED, WAITING)	Ex1, Ex2
5	Reentrancy	(<code>ReentrantLock</code>)	Ex1, Ex2

1 Part 1: Race Conditions

Goal

Understand how concurrent access to shared mutable state without synchronization leads to *lost updates* and broken invariants. Learn the lock-ordering discipline for deadlock-free transfer protocols.

Demos

1.0.1 Demo01 – Unsynchronized Counter Race

What it demonstrates:

- A counter field incremented by two threads without protection
- Interleaving of reads and writes causes lost updates
- Final count is unpredictable; often less than expected total
- Shows why atomicity is necessary

Key insight: Even a single operation like `count++` is not atomic at the machine code level; it compiles to read-modify-write, exposing a race window.

Code pattern:

```
public class Demo01_BrokenCounterRace {
    private int count = 0;

    public void increment() {
        count++; // NOT atomic: read, add 1, write
    }

    // Two threads race here; result varies per run
}
```

1.0.2 Demo02 – Bank Account Transfer Race

What it demonstrates:

- A bank account with `balance` field and `withdraw()` method
- Concurrent withdrawals violate the invariant: `balance >= 0`
- Check-then-act races: after check passes, another thread withdraws before this thread subtracts
- Shows why locks must protect entire check-then-act sequences

Key insight: Invariants are not automatically preserved. A check that passes is not a guarantee if state can change before the subsequent action.

Requirements for exercises:

- Fix `withdrawSafe()` with synchronized method
- Implement `transfer(Account from, Account to, int amt)` with lock-ordering (acquire lock on account with lower id first to prevent deadlock)

Exercises

1.0.3 W3.P1.Ex1 – Fix Bank Account (synchronized method)

Goal: Protect the check-then-act sequence with a monitor.

Requirements:

- Make `withdrawSafe(int amount)` thread-safe using `synchronized` method
- The condition check and the update must be atomic as a unit
- No temporary arrays or complex bookkeeping

1.0.4 W3.P1.Ex2 – Safe Transfer via Lock-Ordering

Goal: Implement a deadlock-free transfer protocol between two accounts.

Requirements:

- Implement static `transfer(Account from, Account to, int amount)`
- Always acquire locks in a consistent order: use `id` field (e.g., lower id first)
- Use nested `synchronized(lock)` blocks
- Prevent deadlock by never acquiring locks in reverse order within any invocation

Hint: Deadlock occurs when threads hold different locks and wait for each other's. By always acquiring in id-order, a global lock order is established, eliminating circular waits.

2 Part 2: Monitors & Lock Splitting

Goal

Understand Java's intrinsic monitor model and how explicit lock objects provide fine-grained control over protected regions. Learn lock-splitting to reduce contention when guarding independent resources.

Demos

2.0.1 Demo03 – Safe Counter with Shared Lock

What it demonstrates:

- An explicit `private final Object lock` field as a shared monitor
- `synchronized(lock)` blocks protecting critical sections
- Compare to Demo01: synchronization enforces happens-before, preventing races
- Shows how a single lock can protect a single shared variable

Key insight: A monitor is identified by object identity, not by naming or scope.

2.0.2 Demo04 – Synchronized Methods vs Blocks

What it demonstrates:

- `synchronized` on method signature locks on `this`
- `synchronized(lock)` blocks lock on an explicit object
- Both mechanisms use the same intrinsic monitor (no hyperscaling of mutex objects)
- The lock is held for the duration of the critical section

Code pattern:

```
public class Demo04_SynchronizedMethodVsBlock {
    private final Object lock = new Object();

    // Pattern A: synchronized method (locks on 'this')
    public synchronized void methodA() { }

    // Pattern B: synchronized block (locks on 'lock')
    public void methodB() {
        synchronized(lock) { }
    }

    // Both patterns are valid; choose based on scope needs
}
```

2.0.3 Demo05 – Safe Bank Account

What it demonstrates:

- A full bank account implementation with synchronized methods
- `deposit()`, `withdraw()`, `getBalance()` all protected by same lock
- Demonstrates that all methods accessing shared state use the same monitor

2.0.4 Demos 06 & 07 – Lock Splitting

What they demonstrate:

- Demo06 (Worker): Fine-grained locking for two independent counters
- Demo07 (Parking): More realistic example: separate locks for available spaces and parked cars
- When state is logically independent, separate locks reduce contention
- Lock splitting enables concurrent increments to different state (e.g., `parking_spaces` and `cars_parked`)
- Tradeoff: more locks to manage, but better parallelism under contention

Key insight: Lock granularity is a design choice. Finer locks improve parallelism but increase complexity; coarser locks simplify reasoning but reduce throughput.

Exercises

2.0.5 W3.P2.Ex1 – Convert Method Synchronization to Block Synchronization

Goal: Refactor a synchronized-method design to use explicit locks.

Requirements:

- Convert `MethodCounter` (synchronized methods on `this`) to `BlockCounter`
- Use `private final Object lock` with `synchronized(lock)` blocks
- Do **not** mark methods as `synchronized`
- Both designs must behave identically under test

2.0.6 W3.P2.Ex2 – Lock Splitting

Goal: Reduce contention by using separate locks for independent resources.

Requirements:

- Replace a single `lock` guarding both `counterA` and `counterB` with `lockA` and `lockB`
- Operations on A must use only `lockA`; operations on B must use only `lockB`
- Never hold both locks simultaneously (violates independence)
- Read operations must also use the appropriate lock for visibility

3 Part 3: Lock Identity

Goal

Understand that a monitor is identified by object *identity*, not by location or naming. A new object on each call defeats coordination; locks must be shared and live for the entire coordination scope.

Key Concepts

Lock Identity Failures:

- **Method-local lock:** `synchronized(new Object()) ...` on each call creates a new monitor; threads cannot coordinate across calls.
- **Instance vs static:** Instance locks coordinate only within one object instance; static locks coordinate across all instances.
- **Accidental public monitor:** Exposing a lock object (e.g., `public Object lock`) allows external code to synchronize on it, breaking encapsulation and risking conflicts.

Exercises

3.0.1 W3.P3.Ex1 – Fix Broken Counter (Instance Lock Identity)

Goal: Recognize and fix a lock-identity bug where synchronization fails due to a new object each call.

Requirements:

- The broken counter creates a lock inside `increment()`
- Move lock to an instance field (`private final Object lock`)
- Keep the `synchronized(lock)` syntax unchanged
- After fix, counter is thread-safe; before, it races badly

3.0.2 W3.P3.Ex2 – Static Shared Lock

Goal: Coordinate across all instances of a class using a class-level lock.

Requirements:

- Declare `private static final Object CLASS_LOCK = new Object()`
- Use `CLASS_LOCK` in both `increment()` and `getCount()`
- All instances share one monitor, so increments serialize across the entire class
- Tradeoff: global bottleneck, but absolute coordination guarantee

4 Part 4: Thread States – Coordination

Goal

Understand Java thread lifecycle states and how coordination primitives (`wait()`, `notifyAll()`) transition threads through states. Learn producer-consumer patterns using condition variables.

Thread States in Java

- **RUNNABLE:** Thread is eligible to run (may or may not be on CPU)
- **BLOCKED:** Thread is waiting to enter a `synchronized` block/method (another thread holds the monitor)
- **WAITING:** Thread called `wait()` and has released the monitor; waiting for `notify()` or `notifyAll()`
- **TIMED_WAITING:** Like **WAITING**, but with a timeout
- **TERMINATED:** Thread's `run()` method returned

Demos

4.0.1 Producer-Consumer with `wait()` / `notifyAll()`

What it demonstrates:

- A bounded buffer guarded by one lock
- Producer waits if buffer is full (`size == capacity`)
- Consumer waits if buffer is empty (`size == 0`)
- Both use `while (condition) wait()` to loop after spurious wakeups
- `notifyAll()` wakes all waiters; not all will proceed (condition may not hold)
- Establishes happens-before between producer and consumer

Key insight: A thread in **WAITING** state has released the monitor. When notified, it re-enters **BLOCKED** (competing to acquire the monitor), then **RUNNABLE** (once acquired). The condition must be rechecked in a loop.

Exercises

4.0.2 W3.P4.Ex1 – Observe **BLOCKED** State

Goal: Write code that captures a thread in the **BLOCKED** state and verify it programmatically.

Requirements:

- Create a holder thread that acquires a shared monitor and holds it for 3 seconds
- Create a contender thread that attempts the same monitor (will block)
- Snapshot the contender's state and assert it is `Thread.State.BLOCKED`
- Implement helper methods: `getLockHolder()`, `getContender()`, `snapshotState()`

4.0.3 W3.P4.Ex2 – Bounded Buffer with `wait()`/`notifyAll()`

Goal: Implement a bounded buffer using intrinsic monitors and condition variables.

Requirements:

- Implement `put(int element)` – blocks if `size == capacity`
- Implement `take()` – returns an element; blocks if empty
- Use `while (condition) wait()` for both producer and consumer
- Producers call `notifyAll()` after `put`; consumers call it after `take`
- Example: `capacity=2`, `put(1)`, `put(2)` blocks; `take()` unblocks and returns 1

Hint: Spurious wakeups can occur; always recheck the condition in a loop, not an if statement.

5 Part 5: Reentrancy

Goal

Understand reentrancy: a thread can acquire a monitor it already holds without deadlock. Master `ReentrantLock` and the try-finally pattern for explicit lock management.

Key Concepts

Reentrancy: Java's intrinsic monitors are *reentrant* – a thread that already holds a lock can re-enter it (e.g., in a recursive call). Each entry increments a reentrant depth counter; each `wait()` or method exit decrements it. When depth reaches 0, the lock is released.

ReentrantLock: An explicit lock class that mirrors intrinsic monitor behavior but allows more control (try-lock, fairness policy, multiple condition variables).

try-finally Pattern: Always pair `lock.lock()` with `lock.unlock()` in a finally block to ensure the lock is released even if an exception occurs.

Demos

5.0.1 Intrinsic Monitor Reentrancy

What it demonstrates:

- A synchronized method that calls another synchronized method
- The same thread re-enters the same monitor without deadlock
- Reentrant depth increments on entry, decrements on exit
- Enables recursive algorithms to be written with straightforward locking

Code pattern:

```
public class Demo_Reentrancy {
    public synchronized void a() {
        // ...
        b(); // Re-enters the same monitor (reentrant depth++)
    }

    public synchronized void b() {
        // ...
        // (reentrant depth-- on method exit)
    }
}
```

5.0.2 ReentrantLock

What it demonstrates:

- Explicit lock management with `lock()`, `unlock()`, `tryLock()`
- try-finally pattern ensures unlocking even on exception
- More flexible than intrinsic monitors (multiple conditions, fairness)
- Same reentrancy semantics: a thread can re-lock a `ReentrantLock` it holds

Code pattern:

```
import java.util.concurrent.locks.ReentrantLock;

public class Demo_ReentrantLock {
    private final ReentrantLock lock = new ReentrantLock();

    public void method() {
        lock.lock();
        try {
            // Protected code
        } finally {
            lock.unlock(); // Always called
        }
    }
}
```

Exercises

5.0.3 W3.P5.Ex1 – Intrinsic Lock Reentrancy Depth

Goal: Verify that intrinsic monitors support reentrancy at arbitrary depth.

Requirements:

- Implement `callChain(int depth)` as a synchronized method
- Each level increments `callCount` and recursively calls `callChain(depth - 1)`
- At depth 0, stop and return
- Assert that the thread never deadlocks and `callCount` equals the initial depth
- Example: `callChain(100)` should re-enter the monitor 100 times without deadlock

5.0.4 W3.P5.Ex2 – ReentrantLock Counter

Goal: Rewrite a thread-safe counter using explicit `ReentrantLock`.

Requirements:

- Declare `private final ReentrantLock lock = new ReentrantLock()`
- Implement `increment()` with `lock.lock(); try ... finally lock.unlock()`
- Implement `getCount()` with the same pattern
- Verify reentrancy: a single thread can call `increment()` recursively without deadlock
- Compare semantics to Part 1 Demo01 (synchronized method version)

Key Takeaways

1. **Race conditions** destroy invariants. Synchronization enforces happens-before edges.
2. **A monitor is identified by object identity.** A new object each call defeats coordination; live monitors must be shared.
3. **Lock ordering** prevents deadlock in multi-lock scenarios. Always acquire locks in a consistent global order.
4. **Thread states** (BLOCKED, WAITING) tell you where a thread is stuck. BLOCKED = competing for a monitor; WAITING = parked in `wait()`.
5. **Condition variables** (`wait()`/`notifyAll()`) coordinate across threads. Always use **while**, not **if**, to recheck conditions after wakeup.
6. **Reentrancy** allows recursive or nested synchronized code without deadlock. Both intrinsic monitors and `ReentrantLock` are reentrant.
7. **try-finally** is essential with explicit locks to ensure cleanup. Intrinsic monitors are exception-safe by design; explicit locks are not.

Next Steps:

- Complete all 10 exercises (2 per part)
- Run `./gradlew studentCheck` to validate
- Review `PITFALLS.md` for common threading mistakes
- See `COURSE_TODO.md` for references to lectures and advanced topics

Resources

GitHub Repository: <https://github.com/maoude/concurrency-course-complete>

LinkedIn Profile: <https://www.linkedin.com/in/mohamadaoude/>

[GitHub](#) [LinkedIn](#)